

From Perceptron to Multilayer Perceptron (MLP)

Comprehensive Educational Walkthrough on Deep Learning Transition & Backprop Mechanics

Author: Laurie MAVOUNGOU (CEO JK AI) | Email: lmavoungou@outlook.be

Executive Summary & Purpose

Transitioning from a single-layer linear model to a deep network is the cornerstone of modern Deep Learning. This guide breaks down the structural, mathematical, and gradient flow evolutions required to upgrade a Softmax Perceptron into a full Multilayer Perceptron (MLP).

Core Objective: Understand how non-linear activations combined with multi-layer error backpropagation enable neural networks to construct high-level feature representations.

1. Conceptual & Architectural Shift

SINGLE-LAYER Direct linear projection:

$$x \in \mathbb{R}^{1 \times d} \rightarrow L = xW + b \rightarrow \text{Softmax}(L) = p$$

Limitation: Bound to a single linear decision hyperplane. Cannot solve non-linearly separable problems like XOR or complex multi-class boundaries.

MULTILAYER (MLP) Cascaded non-linear transformations:

$$x \rightarrow h_1 = \sigma_0(z_0) \rightarrow \dots \rightarrow h_{L-1} \rightarrow L \rightarrow \text{Softmax}(L - \max) = p$$

Superpower: Leveraging the *Universal Approximation Theorem*, hidden layers warp and project input space into linearly separable representations.

2. Key Codebase Upgrades

- Dynamic Topologies:** Replaced fixed single weights with dynamic weight lists initialized via `layer_sizes`.
- Activation Caching (z & h):** Added persistent memory lists to record pre-activations (z) and post-activations (h) during forward passes.
- Alternating Non-Linearities:** Integrated alternating activation functions (ReLU for even layers, Hard Tanh for odd layers).
- Shifted Softmax Stability:** Subtracts `max(L)` prior to exponentiation to eliminate numerical overflow (`inf / NaN`).
- Backward Gradient Flow:** Replaced simple weight updates with recursive backpropagation using intermediate layer deltas (δ_u).

4. Detailed Backpropagation Mechanics

Backpropagation is the application of the multivariate **Chain Rule** to propagate credit/blame for loss backward from output to inputs.

Step 1: Output Layer Loss & Gradient (δ_{out})

For Cross-Entropy loss $L = -\ln(p_y)$, taking partial derivative with respect to final logits L_j gives:

$$\delta_{out} = \partial L / \partial L = p - e_y \quad (\text{where } e_y \text{ is target one-hot vector})$$

Step 2: Backward Propagation across Hidden Layers (δ_u)

To compute loss sensitivity at hidden layer u , error signal from next layer δ_{u+1} is pulled back through transpose weight matrix W_{u+1}^T and modulated element-wise ($\odot$) by the local activation derivative $\sigma'_u(z_u)$:

$$\delta_u = (\delta_{u+1} \cdot W_{u+1}^T) \odot \sigma'_u(z_u)$$

Activation Derivatives Used:

- ReLU Derivative (Even Layers):** $\sigma'(z) = 1$ if $z > 0$ else 0
- Hard Tanh Derivative (Odd Layers):** $\sigma'(z) = 1$ if $-1 < z < 1$ else 0

Step 3: Weight and Bias Gradient Accumulation

With delta signals computed for every layer, parameter gradients are obtained via exterior product with layer inputs (a_{prev}):

$$\partial L / \partial W_u = a_{prev}^T \cdot \delta_u \quad | \quad \partial L / \partial b_u = \delta_u$$

(Where $a_{prev} = x$ for layer $u=0$, and $a_{prev} = h_{u-1}$ for subsequent layers).

Step 4: Gradient Descent Parameter Update Rule

$$W_u \leftarrow W_u - \eta \cdot (\partial L / \partial W_u) \quad | \quad b_u \leftarrow b_u - \eta \cdot \delta_u$$

5. Interactive Student Lab Exercises

- Exercise 1 (Vectorized Softmax):** Replace the explicit Python loop in Softmax normalization with pure 1D/2D NumPy vectorized broadcasting.
- Exercise 2 (Mini-Batch Matrix Support):** Expand the current single-instance formulation ($x \in \mathbb{R}^{1 \times d}$) to process batch matrices ($X \in \mathbb{R}^{B \times d}$).
- Exercise 3 (Optimizer Upgrade):** Replace basic Vanilla Gradient Descent updates with Momentum ($v = \beta v + \eta g$) or Adam optimizer.



3. Structural Comparison

Feature	Single-Layer	Multilayer MLP
Depth	1 (Input → Output)	N (Input → Hidden → Output)
Activations	Output Softmax only	ReLU / Hard Tanh (Hidden) + Softmax
State Memory	w, b	$w_{\text{hidden}}, b_{\text{hidden}}, z, h, \delta$
Softmax	Unstable ($\exp(L)$)	Shifted Stable ($\exp(L - \max)$)
Grad Computation	Single Output Step	Recursive Multi-Layer Chain Rule